

# Recursion

Programming for Problem Solving (PPS)

GTU # 3110003

USING

{C}

Programming



**Prof. Nilesh Gambhava**

Computer Engineering Department,  
Darshan Institute of Engineering & Technology, Rajkot



# What is Recursion?

- ▶ Any function which calls itself is called **recursive function** and such function calls are called **recursive calls**.
- ▶ **Recursion** cannot be applied to all problems, but it is more useful for the tasks that can be defined in terms of a similar subtask.
- ▶ It is idea of representing problem a with smaller problems.
- ▶ Any problem that can be solved **recursively** can be solved iteratively.
- ▶ When **recursive** function call itself, the memory for called function allocated and different copy of the local variable is created for each function call.
- ▶ Some of the problem best suitable for recursion are
  - Factorial
  - Fibonacci
  - Tower of Hanoi

# Working of Recursive function

Working

```
void func1();
```

```
void main()
{
    ....
    func1();
    ....
}
```

```
void func1()
{
    ....
    func1();
    ....
}
```

Function  
call

**Recursive**  
function call

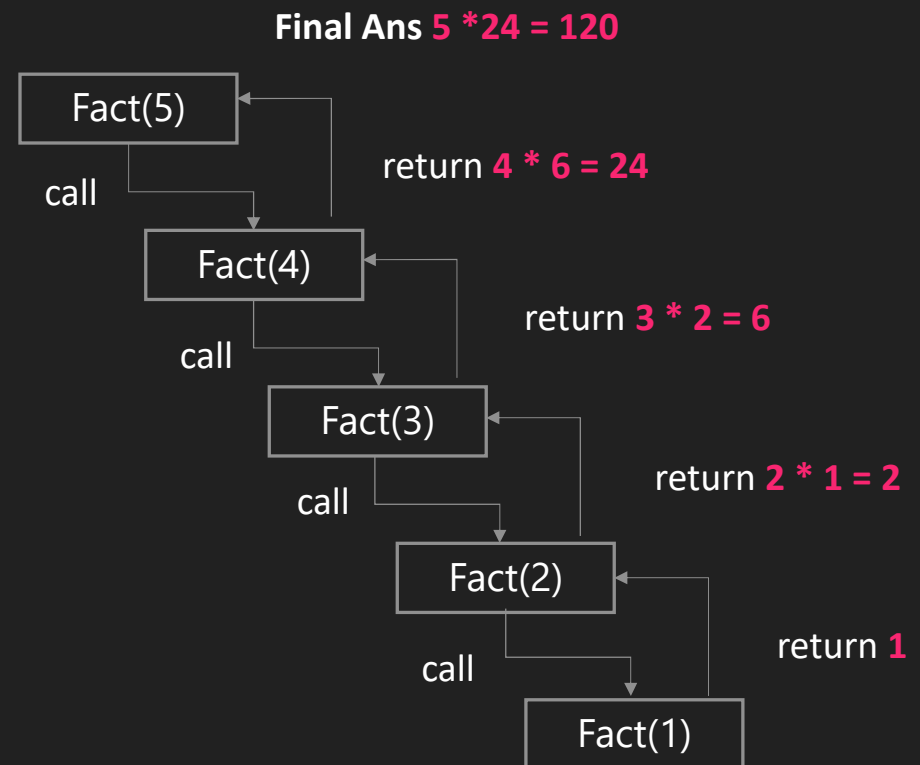
# Properties of Recursion

- ▶ A **recursive** function can go infinite like a loop. To avoid infinite running of recursive function, there are two properties that a recursive function must have.
- ▶ **Base Case or Base criteria**
  - ➔ It allows the recursion algorithm to stop.
  - ➔ A base case is typically a problem that is small enough to solve directly.
- ▶ **Progressive approach**
  - ➔ A recursive algorithm must change its state in such a way that it moves forward to the base case.

# Recursion - factorial example

- ▶ The factorial of a integer  $n$ , is product of
  - ↳  $n * (n-1) * (n-2) * \dots * 1$
- ▶ Recursive definition of factorial
  - ↳  $n! = n * (n-1)!$
  - ↳ Example
    - $3! = 3 * 2 * 1$
    - $3! = 3 * (2 * 1)$
    - $3! = 3 * (2!)$

## Recursive trace



# WAP to find factorial of given number using Recursion

## Program

```
1  #include <stdio.h>
2  int fact(int);
3  void main()
4  {
5      int n, f;
6      printf("Enter the number?\n");
7      scanf("%d", &n);
8      f = fact(n);
9      printf("factorial = %d", f);
10 }
11 int fact(int n)
12 {
13     if (n == 0)
14         return 1;
15     else if (n == 1)
16         return 1;
17     else
18         return n * fact(n - 1);
19 }
```

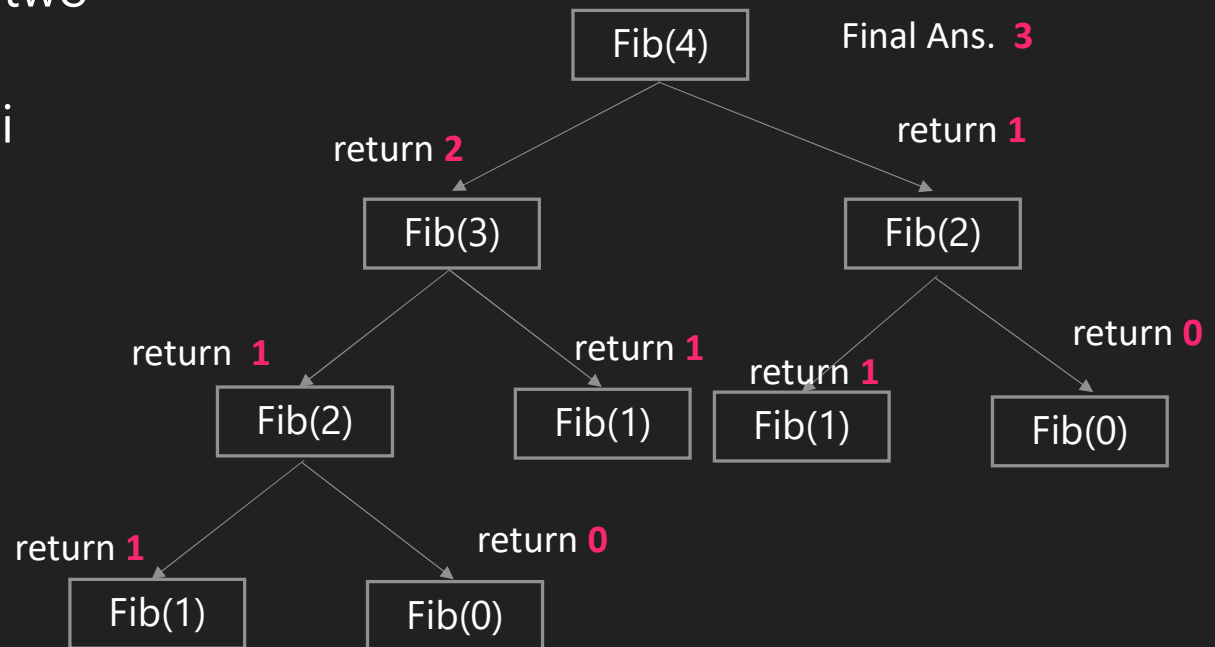
## Output

```
Enter the number? 5
factorial = 120
```

# Recursion - Fibonacci example

- ▶ A series of numbers, where next number is found by adding the two number before it.
- ▶ Recursive definition of Fibonacci
  - $\text{Fib}(0) = 0$
  - $\text{Fib}(1) = 1$
  - $\text{Fib}(n) = \text{Fib}(n-1) + \text{Fib}(n-2)$
- ▶ Example
  - $\text{Fib}(4) = \text{Fib}(3) + \text{Fib}(2)$
  - $\text{Fib}(4) = 3$

## Recursive trace



# WAP to Display Fibonacci Sequence

## Program

```
1  #include <stdio.h>
2  int fibonacci(int);
3  void main()
4  {
5      int n, m = 0, i;
6      printf("Enter Total terms\n");
7      scanf("%d", &n);
8      printf("Fibonacci series\n");
9      for (i = 1; i <= n; i++)
10     {
11         printf("%d ", fibonacci(m));
12         m++;
13     }
14 }
```

## Program contd.

```
15 int fibonacci(int n)
16 {
17     if (n == 0 || n == 1)
18         return n;
19     else
20         return (fibonacci(n - 1) +
21                 fibonacci(n - 2));
22 }
```

## Output

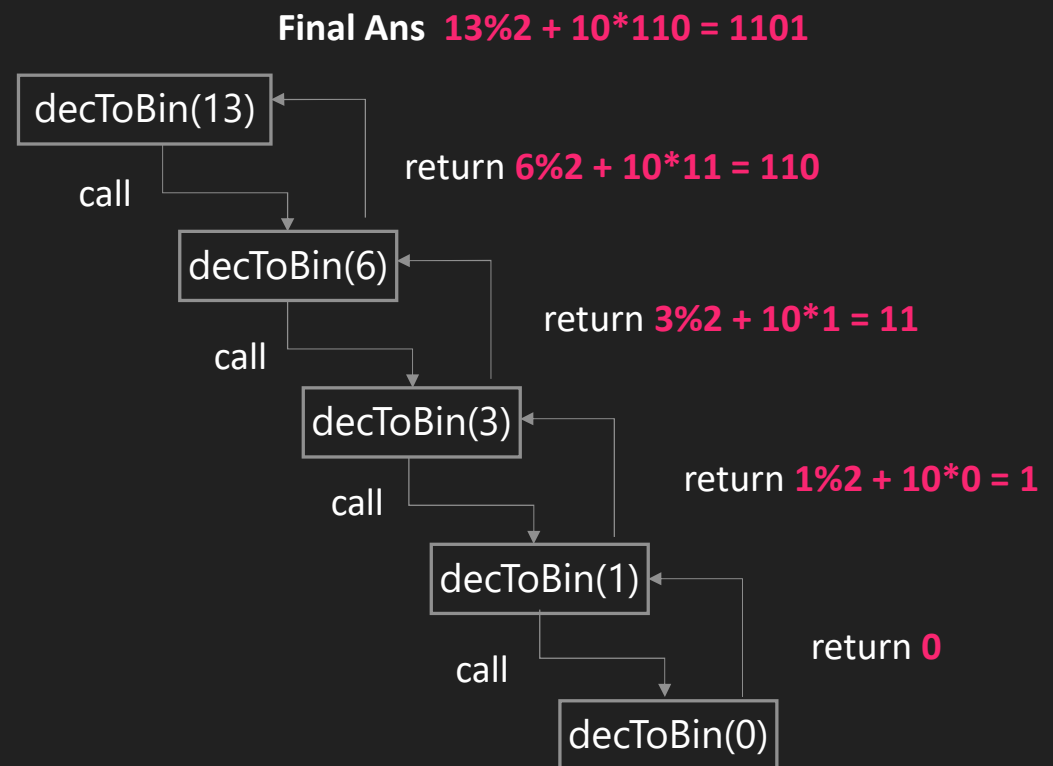
```
Enter Total terms
5
Fibonacci series
0 1 1 2 3
```



# Recursion - Decimal to Binary example

- ▶ To convert decimal to binary, divide decimal number by 2 till dividend will be less than 2
- ▶ To convert decimal 13 to binary
  - $13/2 = 6$  remainder **1**
  - $6/2 = 3$  remainder **0**
  - $3/2 = 1$  remainder **1**
  - $1/2 = 0$  remainder **1**
- ▶ Recursive definition of Decimal to Binary
  - $\text{decToBin}(0) = 0$
  - $\text{decToBin}(n) = n\%2 + 10 * \text{decToBin}(n/2)$
- ▶ Example
  - $\text{decToBin}(13) = 13\%2 + 10 * \text{decToBin}(6)$
  - $\text{decToBin}(13) = 1101$

## Recursive trace



# WAP to Convert Decimal to Binary

## Program

```
1  #include <stdio.h>
2  int convertDecimalToBinary(int);
3  void main()
4  {
5      int dec, bin;
6      printf("Enter a decimal number: ");
7      scanf("%d", &dec);
8      bin = convertDecimalToBinary(dec);
9      printf("The binary equivalent = %d \n", bin);
10 }
11 int convertDecimalToBinary(int dec)
12 {
13     if (dec == 0)
14         return 0;
15     else
16         return (dec % 2 + 10 *
17             convertDecimalToBinary(dec / 2));
17 }
```

## Output

```
Enter a decimal number: 12
The binary equivalent = 1100
```

# WAP to Convert Binary to Decimal

## Program

```
1  #include <stdio.h>
2  int convertBinaryToDecimal(int b, int c, int t);
3  void main()
4  {
5      unsigned int binary, decimal;
6      printf("Enter a binary number: ");
7      scanf("%d", &binary);
8      decimal = convertBinaryToDecimal(binary, 1, 0);
9      printf("Decimal value of %d is %d", binary, decimal);
10 }
11 int convertBinaryToDecimal(int b, int c, int t)
12 {
13     if (b > 0)
14     {
15         t += (b % 10) * c;
16         convertBinaryToDecimal(b / 10, c * 2, t);
17     }
18     else
19         return t;
20 }
```

## Output

```
Enter a binary number: 101
Decimal value of 101 is 5
```

# Practice Programs

- 1) Write a program to find factorial of a given number using recursion.
- 2) WAP to convert decimal number into binary using recursion.
- 3) WAP to use recursive calls to evaluate  $F(x) = x - x^3/3! + x^5/5! - x^7/7! + \dots + x^n/n!$



*Thank you*

